



School of Information Technology and Engineering
Security of Operating System

SIS 4

Working with Filesystems: Building, Monitoring and Supporting Jobs
“Secure SWIFT Logical Terminal Infrastructure”

Checked by: Valyayev Denis

Done by: Faramarz Pedram

Almaty, 2026

Topic

Storage policy definition, redundancy implementation, and backup system configuration in a secure SWIFT logical terminal infrastructure.

Target

The goal of this study is to design and implement a secure and reliable storage system for the Secure SWIFT Logical Terminal Infrastructure. This includes defining a storage policy, implementing a redundancy mechanism, configuring an automated backup system, and verifying data recovery.

1. Introduction

This study is a continuation of the previous stages of the project:

- SIS 1 — The system architecture was designed with two machines: swift-terminal (main SWIFT logical terminal running Nginx, FastAPI backend, and PostgreSQL) and swift-support (central logs, backups, and monitoring).
- SIS 2 — Filesystem structure, users, roles, and secure directories such as /swift-lt were created with strict ownership (swiftapi:swift-ops) and permissions (750). LUKS disk encryption was applied to protect sensitive SWIFT data.
- SIS 3 — Required packages and services were installed and configured on both machines: Nginx, FastAPI backend, PostgreSQL, rsyslog, Grafana, and Node Exporter. Firewall rules were applied to allow only required ports.

In SIS 4, the focus shifts to data protection, availability, and recovery — critical requirements for a financial system handling SWIFT messages.

2. Storage Policy Design

2.1 Sensitive Data Storage

- Location: /swift-lt/ on swift-terminal

- Contains SWIFT logical terminal files (LT1–LT5)
- Protected by strict permissions (750), dedicated service account (swiftapi), group-based access (swift-ops), and LUKS disk encryption configured in SIS 2

2.2 System and Application Data

- PostgreSQL data: /var/lib/pgsql/data/
- Application code: /opt/swift-backend/
- Local logs: /var/log/ on swift-terminal
- Remote logs: /var/log/remote/ on swift-support

2.3 Backup Storage

- Location: /backup/swift/ on swift-support
- Access restricted to backup user with permissions 700

2.4 Storage Layout Summary

Machine	Path	Purpose	Access
swift-terminal	/swift-lt/LT1–LT5	SWIFT file storage (most sensitive)	swiftapi:swift-ops, 750
swift-terminal	/var/lib/pgsql/data/	PostgreSQL metadata database	postgres user only
swift-terminal	/opt/swift-backend/	Backend application code	swiftapi user

swift-terminal	/var/log/	Local system and service logs	root / rsyslog
swift-support	/backup/swift/	Backup repository	backup user, 700
swift-support	/var/log/remote/	Centralized log storage	rsyslog

2.5 Policy Principles

- Separation of data between main server and backup server
- Least privilege — no direct user access to sensitive files
- No direct user access to sensitive data
- Centralized backup storage on a dedicated machine
- Encrypted storage for critical SWIFT data

2.6 Fail-Safe Strategy

Risk	Mitigation	Layer
Single disk failure	RAID 1 mirror on /swift-lt volume	Hardware/RAID
Data deletion or corruption	Daily encrypted backups to swift-support	Backup
Ransomware or unauthorized modification	Separate backup server, immutable logs	Isolation
System crash	ext4 journaling + journald for crash recovery	OS

Loss of audit trail	Centralized rsyslog on swift-support	Logging
---------------------	--------------------------------------	---------

3. Redundancy and Availability

3.1 Redundancy Strategy

The redundancy strategy uses two complementary layers:

- RAID 1 (Mirroring) — Implemented in Section 3.3. Two virtual disks mirror each other to protect against single disk failure on swift-terminal.
- Remote Backup Server — Regular secure replication of data to swift-support. This protects against disk failure, ransomware, accidental deletion, and full machine loss.

While RAID provides disk-level redundancy, it does not protect against ransomware, accidental deletion, or full machine failure. The combination of RAID 1 and remote backup provides the strongest guarantee for this infrastructure.

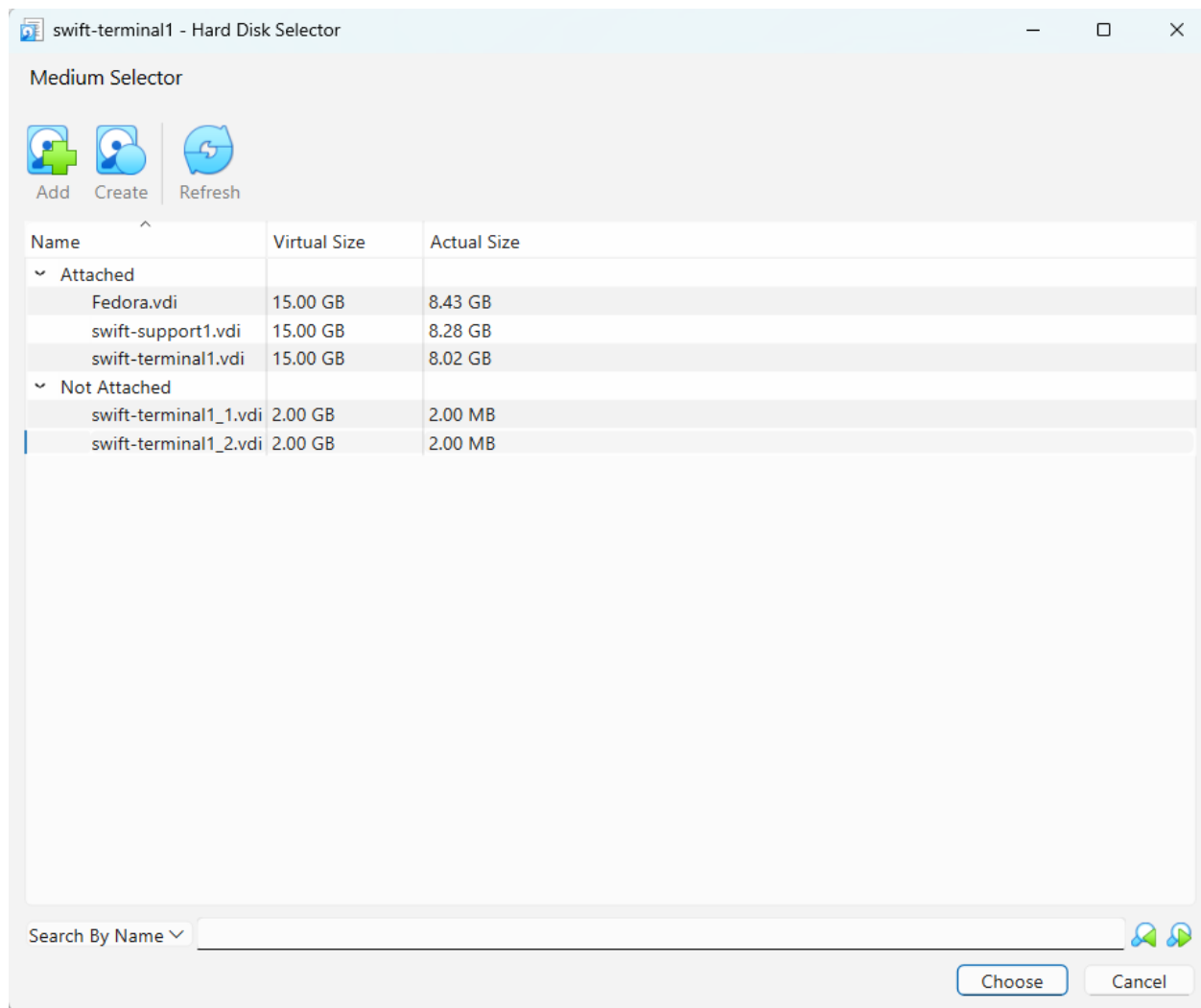
3.2 Justification

- Protects against disk failure on the main server
- Protects against ransomware or full system compromise
- Ensures data availability even if swift-terminal is completely lost
- Remote backup server is isolated — a compromise of swift-terminal does not automatically affect swift-support

3.3 RAID 1 Implementation

RAID level chosen: RAID 1 (Mirroring)

Two additional virtual disks (sdb and sdc, 2 GB each) were added to swift-terminal in VirtualBox settings to simulate a two-disk RAID array.



Verify new disks are visible:

```
swift-terminal@fedora:~/Desktop$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   15G  0 disk
├─sda1       8:1    0    1M  0 part
├─sda2       8:2    0    2G  0 part /boot
└─sda3       8:3    0   13G  0 part /home
              /
sdb          8:16   0    2G  0 disk
sdc          8:32   0    2G  0 disk
sr0         11:0    1 1024M  0 rom
zram0       251:0    0   3.9G  0 disk [SWAP]
swift-terminal@fedora:~/Desktop$
```

Install mdadm:

```
swift-terminal@fedora:~/Desktop$ sudo dnf install mdadm -y
[sudo] password for swift-terminal:
Updating and loading repositories:
  Fedora 43 - x86_64 - Update 100% | 2.2 KiB/s | 5.9 KiB | 00m03s
  Fedora 43 - x86_64          100% | 4.7 KiB/s | 8.4 KiB | 00m02s
  Fedora 43 openh264 (From Ci 100% | 1.3 KiB/s | 986.0 B | 00m01s
Repositories loaded.
Package "mdadm-4.3-9.fc43.x86_64" is already installed.

Nothing to do.
swift-terminal@fedora:~/Desktop$
```

Create the RAID 1 array:

```
swift-terminal@fedora:~/Desktop$ sudo mdadm --create /dev/md0 --level
=1 --raid-devices=2 /dev/sdb /dev/sdc
[sudo] password for swift-terminal:
mdadm: Note: this array has metadata at the start and
      may not be suitable as a boot device.  If you plan to
      store '/boot' on this device please ensure that
      your boot-loader understands md/v1.x metadata, or use
      --metadata=0.90
Continue creating array?
Continue creating array? (y/n) y
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

Monitor synchronization progress:

```
swift-terminal@fedora:~/Desktop$ cat /proc/mdstat
Personalities : [raid1]
md0 : active raid1 sdc[1] sdb[0]
      2094080 blocks super 1.2 [2/2] [UU]

unused devices: <none>
swift-terminal@fedora:~/Desktop$
```

Check array details, format the RAID device with ext4:

```
swift-terminal@fedora:~/Desktop$ sudo mdadm --detail /dev/md0  
/dev/md0:
```

```
    Version : 1.2  
    Creation Time : Fri Mar 27 20:33:48 2026  
    Raid Level : raid1  
    Array Size : 2094080 (2045.00 MiB 2144.34 MB)  
    Used Dev Size : 2094080 (2045.00 MiB 2144.34 MB)  
    Raid Devices : 2  
    Total Devices : 2  
    Persistence : Superblock is persistent  
  
    Update Time : Fri Mar 27 20:33:58 2026  
    State : clean  
    Active Devices : 2  
    Working Devices : 2  
    Failed Devices : 0  
    Spare Devices : 0
```

```
Consistency Policy : resync
```

```
    Name : fedora:0 (local to host fedora)  
    UUID : 97c430c8:660af760:c3e599f9:6a432e4a  
    Events : 17
```

Number	Major	Minor	RaidDevice	State	
0	8	16	0	active sync	/dev/sdb
1	8	32	1	active sync	/dev/sdc

```
swift-terminal@fedora:~/Desktop$
```

```
swift-terminal@fedora:~/Desktop$ sudo mkfs.ext4 /dev/md0  
mke2fs 1.47.3 (8-Jul-2025)  
Creating filesystem with 523520 4k blocks and 131072 inodes  
Filesystem UUID: b0031bcf-e828-4e73-ac22-74d75fd47171  
Superblock backups stored on blocks:  
    32768, 98304, 163840, 229376, 294912
```

```
Allocating group tables: done  
Writing inode tables: done  
Creating journal (8192 blocks): done  
Writing superblocks and filesystem accounting information: done
```

Mount the RAID array and make it persistent:


```

swift-terminal@fedora:~/Desktop$ sudo mkdir -p /swift-raid
swift-terminal@fedora:~/Desktop$ sudo mount /dev/md0 /swift-raid
swift-terminal@fedora:~/Desktop$ df -h /swift-raid
Filesystem      Size  Used Avail Use% Mounted on
/dev/md0        2.0G  532K  1.9G   1% /swift-raid
swift-terminal@fedora:~/Desktop$ echo '/dev/md0 /swift-raid ext4 default 0 2' | sudo tee -a /etc/fstab
/dev/md0 /swift-raid ext4 default 0 2

```

Save mdadm configuration for boot-time assembly, apply ownership and permissions:

```

swift-terminal@fedora:~/Desktop$ sudo mdadm --detail --scan | sudo tee -a /etc/mdadm.conf
ARRAY /dev/md0 metadata=1.2 UUID=97c430c8:660af760:c3e599f9:6a432e4a
swift-terminal@fedora:~/Desktop$ sudo dracut -H -f

swift-terminal@fedora:~/Desktop$ sudo chown -R swiftapi:swift-ops /swift-raid
swift-terminal@fedora:~/Desktop$ sudo chmod -R 750 /swift-raid/
swift-terminal@fedora:~/Desktop$ ls -ld /s
sbin/      srv/      swift-lt/  swift-raid/ sys/
swift-terminal@fedora:~/Desktop$ ls -ld /s
sbin/      srv/      swift-lt/  swift-raid/ sys/
swift-terminal@fedora:~/Desktop$ ls -ld /swift-raid
drwxr-x--- 3 swiftapi swift-ops 4096 Mar 27 20:36 /swift-raid
swift-terminal@fedora:~/Desktop$

```

3.4 RAID Failure Simulation Test

Mark one disk as failed to verify the array continues in degraded mode:

```

swift-terminal@fedora:~/Desktop$ sudo mdadm --fail /dev/md0 /dev/sdb
mdadm: set /dev/sdb faulty in /dev/md0
swift-terminal@fedora:~/Desktop$ cat /proc/mdstat
Personalities : [raid1]
md0 : active raid1 sdc[1] sdb[0](F)
      2094080 blocks super 1.2 [2/1] [_U]

unused devices: <none>

```

Result: Array is active (degraded). Data remains accessible. Re-add the disk to trigger automatic rebuild:

```

swift-terminal@fedora:~/Desktop$ sudo mdadm --remove /dev/md0 /dev/sdb
mdadm: hot removed /dev/sdb from /dev/md0
swift-terminal@fedora:~/Desktop$ sudo mdadm --add /dev/md0 /dev/sdb
mdadm: added /dev/sdb
swift-terminal@fedora:~/Desktop$ cat /proc/mdstat
Personalities : [raid1]
md0 : active raid1 sdb[2] sdc[1]
      2094080 blocks super 1.2 [2/1] [_U]
      [=====>....]  recovery = 83.8% (1756480/2094080) finish=0.0min speed=219560K/sec

unused devices: <none>
swift-terminal@fedora:~/Desktop$

```

The array rebuilds automatically, confirming RAID 1 redundancy is working correctly.

4. Backup Mechanism Implementation

4.1 Backup Scope

- /swift-lt/ — SWIFT logical terminal files (LT1–LT5)
- PostgreSQL swiftdb — metadata database dump

4.2 Tools Used

- tar — archiving and compression
- rsync — secure transfer to swift-support
- cron — automation and scheduling
- SSH keys — passwordless authentication for automated transfers

4.3 Backup Storage Location

- Destination: /backup/swift/ on swift-support
- Backup user has restricted permissions (700) — no other user can access backup files
- Retention: 7 daily backups, archives older than 7 days are deleted automatically

4.4 SSH Key Setup

Enable ssh on Swift-support:

```
swift-support@fedora:/$ sudo systemctl start sshd
swift-support@fedora:/$ sudo systemctl enable sshd
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
swift-support@fedora:/$ systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; >
   Drop-In: /usr/lib/systemd/system/service.d
           └─10-timeout-abort.conf
   Active: active (running) since Fri 2026-03-27 20:52:26 +05; 12s>
 Invocation: 6db120ebbd85476e900246383531230f
   Docs: man:sshd(8)
         man:ssh_config(5)
 Main PID: 3367 (sshd)
   Tasks: 1 (limit: 4784)
  Memory: 1.2M (peak: 1.4M)
    CPU: 13ms
   CGroup: /system.slice/ssh.service
           └─3367 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 >

Mar 27 20:52:26 fedora systemd[1]: Starting sshd.service - OpenSSH s>
Mar 27 20:52:26 fedora sshd[3367]: Server listening on 0.0.0.0 port >
Mar 27 20:52:26 fedora sshd[3367]: Server listening on :: port 22.
Mar 27 20:52:26 fedora systemd[1]: Started sshd.service - OpenSSH se>
lines 1-19/19 (END)
```

SSH key-based authentication was configured between swift-terminal and swift-support to allow automated password less transfers:

```

swift-terminal@fedora:~/Desktop$ sudo -u admin1 ssh-keygen -t ed25519 -C
'backup@swift-terminal' -f /home/admin1/.ssh/backup_key -N ''
Generating public/private ed25519 key pair.
Created directory '/home/admin1/.ssh'.
Your identification has been saved in /home/admin1/.ssh/backup_key
Your public key has been saved in /home/admin1/.ssh/backup_key.pub
The key fingerprint is:
SHA256:JFXT6qUZoLJjqgoz4UERB/9j+TbbN/aYxhDyUxLw/Cg backup@swift-terminal
The key's randomart image is:
+--[ED25519 256]--+
| oo.   .o.o.   |
| .o   .+.  ..  |
| ..   ...+..   |
| .  o o+ o=..   |
| .    B ESo+*   |
| ..  = o .++    |
| ...o . +  +    |
| o..   . +  *o   |
| *+    . .+oo.   |
+-----[SHA256]-----+

```

```

swift-terminal@fedora:~/Desktop$ ssh-copy-id -i ~/.ssh/backup
_key.pub backup1@192.168.56.102
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/home/swift-terminal/.ssh/backup_key.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new
key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -
- if you are prompted now it is to install the new keys
backup1@192.168.56.102's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /home/swift-t
erminal/.ssh/backup_key 'backup1@192.168.56.102'"
and check to make sure that only the key(s) you wanted were a
dded.

```

Test the connection:

```
swift-terminal@fedora:~/Desktop$ ssh -i ~/.ssh/backup_key bac  
kup1@192.168.56.102 'echo Connection OK'  
Connection OK  
swift-terminal@fedora:~/Desktop$
```

4.5 Creating the PostgreSQL Database

Create the swiftdb database that the backup script will dump:

```
swift-terminal@fedora:~/Desktop$ sudo -u postgres psql  
[sudo] password for swift-terminal:  
psql (18.1)  
Type "help" for help.  
  
postgres=# CREATE DATABASE swiftdb;  
\q  
CREATE DATABASE  
swift-terminal@fedora:~/Desktop$
```

4.6 Backup Script

The backup script was created at /usr/local/bin/swift-backup.sh:

```
Mar 27 10:47 PM
swift-terminal@fedora:~/Desktop — sudo nano /usr/local/bin/swift-backup.sh
GNU nano 8.5 /usr/local/bin/swift-backup.sh Modified
#!/bin/bash

# Secure SWIFT Backup Script

DATE=$(date +%F)
SSH_KEY=/home/admin1/.ssh/backup_key
REMOTE_USER=backup1
REMOTE_HOST=192.168.56.102
REMOTE_PATH=/backup/swift/

# Archive SWIFT LT files
tar -czf /tmp/swift-lt-$DATE.tar.gz /swift-lt

# Dump PostgreSQL database
sudo -u postgres pg_dumpall > /tmp/db-$DATE.sql

# Transfer archives to swift-support
sudo -u admin1 rsync -avz -e "ssh -i $SSH_KEY" /tmp/swift-lt-$DATE.ta>
$REMOTE_USER@$REMOTE_HOST:$REMOTE_PATH

sudo -u admin1 rsync -avz -e "ssh -i $SSH_KEY" /tmp/db-$DATE.sql \
$REMOTE_USER@$REMOTE_HOST:$REMOTE_PATH

# Cleanup local temp files
rm -f /tmp/swift-lt-$DATE.tar.gz /tmp/db-$DATE.sql

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Make the script executable:

```
swift-terminal@fedora:~/Desktop$ sudo chmod +x /usr/local/bin/swi
ft-backup.sh
swift-terminal@fedora:~/Desktop$
```

4.7 Scheduling with Cron

The backup job was scheduled to run daily at 02:00:

```
swift-terminal@fedora:~/Desktop — sudo crontab -e
~/Desktop
0 2 * * * /usr/local/bin/swift-backup.sh >> /var/log/swift-backup.log 2
>&1
~
~
~
~
```

Verify the entry:

```
swift-terminal@fedora:~/Desktop$ sudo crontab -l
0 2 * * * /usr/local/bin/swift-backup.sh >> /var/log/swift-backup.log
2>&1
```

4.8 Backup Verification

After running the script manually, the backup directory on swift-support was checked:

```
swift-terminal@fedora:~/Desktop$ sudo /usr/local/bin/swift-backup.sh
tar: Removing leading '/' from member names
sending incremental file list
swift-lt-2026-03-27.tar.gz

sent 319 bytes  received 35 bytes  236.00 bytes/sec
total size is 193  speedup is 0.55
sending incremental file list
db-2026-03-27.sql

sent 1,025 bytes  received 35 bytes  706.67 bytes/sec
total size is 3,463  speedup is 3.27
Backup completed: 2026-03-27
```

Both the .tar.gz archive and the .sql database dump were confirmed present, verifying that file data and metadata are properly backed up.

```
backup1@fedora:/$ ls -lh /backup/swift/
total 8.0K
-rw-r--r--. 1 backup1 backup1 3.4K Mar 27 22:44 db-2026-03-27.sql
-rw-r--r--. 1 backup1 backup1 193 Mar 27 22:44 swift-lt-2026-03-27.t
ar.gz
backup1@fedora:/$
```

5. Monitoring Storage and Backup

Basic monitoring was implemented to track disk usage and verify backup health:

Check disk usage on both machines:

```
swift-terminal@fedora:~/Desktop$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3        13G   6.6G   6.4G   51% /
devtmpfs         2.0G     0   2.0G    0% /dev
tmpfs            2.0G   1.2M   2.0G    1% /dev/shm
tmpfs            807M   1.5M   806M    1% /run
tmpfs            1.0M     0   1.0M    0% /run/credentials/systemd-journald.service
tmpfs            2.0G    28K   2.0G    1% /tmp
/dev/sda3        13G   6.6G   6.4G   51% /home
/dev/sda2        2.0G  371M   1.5G   21% /boot
tmpfs            1.0M     0   1.0M    0% /run/credentials/systemd-resolved.service
tmpfs            404M   148K   404M    1% /run/user/1000
/dev/md0         2.0G   532K   1.9G    1% /swift-raid
tmpfs            404M    48K   404M    1% /run/user/0
swift-terminal@fedora:~/Desktop$ S
```

```
swift-support@fedora:/$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3        13G   6.5G   5.9G   53% /
devtmpfs         2.0G     0   2.0G    0% /dev
tmpfs            2.0G   100K   2.0G    1% /dev/shm
tmpfs            807M   1.8M   806M    1% /run
tmpfs            1.0M     0   1.0M    0% /run/credentials/systemd-journald.service
tmpfs            2.0G    16K   2.0G    1% /tmp
/dev/sda3        13G   6.5G   5.9G   53% /home
/dev/sda2        2.0G  374M   1.5G   21% /boot
tmpfs            1.0M     0   1.0M    0% /run/credentials/systemd-resolved.service
tmpfs            404M   140K   404M    1% /run/user/1000
tmpfs            404M   124K   404M    1% /run/user/1004
tmpfs            404M     0   404M    0% /run/user/970
swift-support@fedora:/$
```

Verify backup archives are present on swift-support:


```
backup1@fedora:/$ ls -lh /backup/swift/
total 8.0K
-rw-r--r--. 1 backup1 backup1 3.4K Mar 27 22:44 db-2026-03-27.sql
-rw-r--r--. 1 backup1 backup1 193 Mar 27 22:44 swift-lt-2026-03-27.tar.gz
backup1@fedora:/$
```

Monitor RAID array health:

```
swift-terminal@fedora:~/Desktop$ cat /proc/mdstat
sudo mdadm --detail /dev/md0
Personalities : [raid1]
md0 : active raid1 sdb[2] sdc[1]
      2094080 blocks super 1.2 [2/2] [UU]

unused devices: <none>
/dev/md0:
       Version : 1.2
  Creation Time : Fri Mar 27 20:33:48 2026
    Raid Level : raid1
   Array Size : 2094080 (2045.00 MiB 2144.34 MB)
  Used Dev Size : 2094080 (2045.00 MiB 2144.34 MB)
   Raid Devices : 2
  Total Devices : 2
 Persistence : Superblock is persistent

       Update Time : Fri Mar 27 20:43:34 2026
         State : clean
   Active Devices : 2
 Working Devices : 2
  Failed Devices : 0
   Spare Devices : 0
```

```
Consistency Policy : resync
```

```
      Name : fedora:0 (local to host fedora)
```

```
      UUID : 97c430c8:660af760:c3e599f9:6a432e4a
```

```
      Events : 39
```

Number	Major	Minor	RaidDevice	State		
2	8	16	0	active sync	/dev/sdb	
1	8	32	1	active sync	/dev/sdc	

Check backup log for errors:

```
cat /var/log/swift-backup.log
```

6. Security Considerations

- Backups are stored on a physically separate machine (swift-support) — a compromise of swift-terminal does not expose the backup data
- Backup user has restricted permissions (700) — no other user can read or modify backup archives
- No direct user access to sensitive SWIFT data in /swift-lt/
- Encrypted storage (LUKS) for critical SWIFT data configured in SIS 2
- SSH key authentication for all automated transfers — no passwords stored in scripts
- RAID 1 ensures disk-level redundancy independent of the backup server

7. Backup Recovery Testing

7.1 Test Plan

- Step 1: Create test SWIFT files in /swift-lt/LT1/
- Step 2: Run backup to swift-support

- Step 3: Delete the files to simulate data loss
- Step 4: Restore files and database from backup archives
- Step 5: Verify restored files match originals and permissions are correct

7.2 Step 1 — Create Test Data

```
swift-terminal@fedora:~/Desktop$ sudo -u swiftapi bash -c 'echo "SWIFT-MSG-001: Transfer 500000 USD" > /swift-lt/LT1/msg001.txt'
sudo -u swiftapi bash -c 'echo "SWIFT-MSG-002: Transfer 250000 EUR" > /swift-lt/LT1/msg002.txt'
sudo -u swiftapi ls -l /swift-lt/LT1/
[sudo] password for swift-terminal:
total 8
-rw-r--r--. 1 swiftapi swiftapi 35 Mar 27 23:11 msg001.txt
-rw-r--r--. 1 swiftapi swiftapi 35 Mar 27 23:11 msg002.txt
swift-terminal@fedora:~/Desktop$
```

7.3 Step 2 — Run Backup

```
swift-terminal@fedora:~/Desktop$ sudo /usr/local/bin/swift-backup.sh
ls -lh /backup/swift/
tar: Removing leading `/' from member names
sending incremental file list
swift-lt-2026-03-27.tar.gz

sent 444 bytes  received 41 bytes  194.00 bytes/sec
total size is 317  speedup is 0.65
sending incremental file list
db-2026-03-27.sql

sent 1,022 bytes  received 65 bytes  724.67 bytes/sec
total size is 3,463  speedup is 3.19
Backup completed: 2026-03-27
ls: cannot access '/backup/swift/': No such file or directory
swift-terminal@fedora:~/Desktop$
```

7.4 Step 3 — Simulate Data Loss

```
swift-terminal@fedora:~/Desktop$ sudo -u swiftapi rm -rf /swift-lt/*
swift-terminal@fedora:~/Desktop$ ls /swift-ls/LT1
ls: cannot access '/swift-ls/LT1': No such file or directory
```

The directory is now empty — data has been lost.

7.5 Step 4 — Restore from Backup

Restore files:

```
swift-support@fedora:/$ sudo ls -lh /backup/swift/
[sudo] password for swift-support:
total 8.0K
-rw-r--r--. 1 backup1 backup1 3.4K Mar 27 23:15 db-2026-03-27.sql
-rw-r--r--. 1 backup1 backup1 317 Mar 27 23:15 swift-lt-2026-03-27.tar.gz
```

```
swift-terminal@fedora:~/Desktop$ ssh admin1@192.168.56.102
admin1@192.168.56.102's password:
```

```
admin1@fedora:~$ sudo tar -xzf /backup/swift/swift-lt-2026-03-27.tar.gz -C /
[sudo] password for admin1:
admin1@fedora:~$
```

Restore database:

```
psql < db-YYYY-MM-DD.sql
```

7.6 Step 5 — Verify Restoration

```
swift-terminal@fedora:~/Desktop$ sudo -u swiftapi ls /swift-lt/LT1
[sudo] password for swift-terminal:
msg001.txt  msg002.txt
swift-terminal@fedora:~/Desktop$ sudo -u swiftapi cat /swift-lt/LT1/m
sg001.txt
SWIFT-MSG-001: Transfer 500000 USD
swift-terminal@fedora:~/Desktop$ sudo -u swiftapi cat /swift-lt/LT1/m
sg002.txt
SWIFT-MSG-002: Transfer 250000 EUR
swift-terminal@fedora:~/Desktop$
```

Conclusions

In this study, a complete storage and backup system was designed and implemented for the Secure SWIFT Logical Terminal Infrastructure.

A clear storage policy was defined, separating sensitive SWIFT data, system data, and backups across different locations and machines. The policy enforces least privilege access, encrypted storage for critical data, and centralized backup on a dedicated server.

A RAID 1 software mirror was created using mdadm with two additional virtual disks on swift-terminal, providing hardware-level redundancy for the SWIFT storage volume. A disk failure was simulated and confirmed that the array continued operating in degraded mode and recovered automatically after the disk was re-added.

A reliable backup mechanism was implemented using tar, rsync, and cron, ensuring automated and secure daily transfer of SWIFT files and database dumps to the swift-support server. SSH key-based authentication was configured to allow passwordless automated transfers. Backups older than 7 days are removed automatically to manage storage.

A full recovery test was successfully performed, confirming that the system can restore both file data and the database after complete data loss, with correct ownership and permissions re-applied. The storage policy, RAID redundancy, and

backup system together form a resilient, layered data protection strategy consistent with the defense-in-depth architecture established in SIS 1.